

ADA 3.0

Adaptive Demand Allocation Architecture

Pointer-Driven Ladder Simplification, Decimal Data Convention,
and the Resolved Need Scheduler / Oil Scheduler Logic

A Ground-Up Simplification Pass over ADA 2.0.1 and the V1.1–V2.0
As-Built Ladder — Nebico Oil Melting/Retention System

Author	Sambridda Ranabhat
Organisation	Mechatronics Engineering Pvt. Ltd. (MEPL)
Location	Kathmandu, Nepal
Date	September 9, 2026
Document	Architecture Specification, Version 3.0
Status	Final — Locked

This document is the complete, self-contained description of ADA 3.0: the decimal data representation convention, pointer-driven Sensor Data Acquisition and Averaging, the resolved Sensor Rate Error band-check, the Dynamic Water Target law, PO Designation, the resolved PO Timer Calculation, the finalised Need Scheduler (single indexed arbiter loop with sentinel-based exclusion), and the redesigned Oil Scheduler with closed-loop Modbus failure handling. It supersedes the ladder-level detail of ADA Architecture 2.0.1 and the V1.1/V2.0 as-built program while carrying forward every control law, scoring formula, and thermal relationship that ADA 2.0.1 established at the architecture level.

Contents

1	Scope and Relationship to ADA 2.0.1	3
2	Data Representation Convention	3
2.1	Decimal Convention by Quantity	3
2.2	Comparison Tolerance	4
3	Sensor Data Acquisition	4
3.1	Acquisition Loops	4
3.2	Level Referencing	4
4	Sensor Averaging	5
5	Sensor Rate Error Calculations — Resolved	5
6	Dynamic Water Target	6
7	PO Designation	6
8	PO Timer Calculation — Resolved	7
9	Need Scheduler — Finalised	7
9.1	Scoring — Confirmed Unchanged	7
9.2	Exclusion via a Sentinel, Not a Branch	7
9.3	One Loop, Direction Gated by M4	8
9.4	Tie-Break	8
9.5	On the Jitter	8
9.6	Output	8
10	Oil Scheduler	9
10.1	Per-Node Hardware	9
10.2	Request Flow	9
10.3	Reset	9
10.4	5-Try Modbus Failure Handling	9
11	Relationship to ADA 2.0.1's Fault and Oil-Line Scope	10

Executive Summary

ADA 3.0 is not a re-derivation of the control philosophy already fixed by ADA Architecture 2.0.1 — the thermal control law, the tank scoring formula, and the water/oil separation all carry forward unchanged. What ADA 3.0 does is close the gap between that architecture and a ladder program that can actually be maintained: it replaces repeated per-channel rungs with pointer-driven, indexed loops wherever the underlying channels are structurally identical, and it retires the old mixed float/ $\times 10$ fixed-point number convention in favour of one consistent decimal representation used everywhere.

Two blocks that were left open in earlier revisions are resolved outright in this document: the **Sensor Rate Error** check (now a two-sided band per channel, flow excluded and handled real-time) and the **Need Scheduler arbiter** (now a single indexed loop, direction-gated by the PO-designation flag M4, using a sentinel value to exclude the designated tank instead of a branch). The **Oil Scheduler** is a full redesign around the two outstanding node-level HMI buttons (Start/Stop, Reset) and closes the one deadlock risk in the original Modbus polling scheme: a hard release of the line-occupancy lock, unconditionally, on the fifth failed poll.

Program order is fixed at eight functional blocks, each pointer-driven where the channel structure allows it:

Functional Block

- 1 Sensor Data Acquisition
 - 2 Sensor Averaging
 - 3 Sensor Rate Error Calculations
 - 4 Dynamic Water Target
 - 5 PO Designation
 - 6 PO Timer Calculation
 - 7 Need Scheduler
 - 8 Oil Scheduler
-

Status. Every open item carried over from ADA 2.0.1 and the V1.1/V2.0 ladder review is closed as of this revision. ADA 3.0 is final and locked; no further architectural decisions remain before PLC implementation.

1 Scope and Relationship to ADA 2.0.1

ADA 3.0 is a ground-up simplification pass over ADA 2.0.1 and the as-built V1.1–V2.0 ladder, aimed at two things only:

- (i) replacing repetitive, per-channel rungs with pointer-driven loops wherever the channels involved are structurally identical — same conversion, same averaging pattern, same rate check, just a different physical sensor behind each one; and
- (ii) moving the whole program onto a single, consistent decimal data convention, retiring the old mixed float / $\times 10$ fixed-point approach that required a mental scaling correction depending on which register was being read.

This is a ladder-and-data-representation simplification, not a change to the control philosophy. The thermal control law $T_{\text{target}}(d)$, the tank scoring formula, the water/oil separation, and the fault-supervision scope established in ADA Architecture 2.0.1 (Sections 3, 9, and 12 of that document) are all carried forward unchanged in substance — only their number format and, in the case of the Need Scheduler and Oil Scheduler, their internal implementation, change here.

Reading this document alongside ADA 2.0.1. Where a block’s governing formula or rule was already fixed at the architecture level (Dynamic Water Target, tank scoring, PO Timer’s underlying physics), this document states the rule again for completeness and notes explicitly that only the number format has changed. Where a block was previously left open or under-specified (Sensor Rate Error bounds, the Need Scheduler arbiter mechanism, the Oil Scheduler’s failure handling), this document is the resolution.

2 Data Representation Convention

This is new in ADA 3.0. The old program mixed two conventions: true float math for most sensor processing, but $\times 10$ fixed-point integers for several setpoints and thresholds (e.g. K700/K400 meaning 70.0 °C/40.0 °C, K200/K360 as scaled level bands). ADA 3.0 drops the fixed-point half entirely.

2.1 Decimal Convention by Quantity

Quantity	Convention
Temperature	Real decimal values throughout, held and compared to 1 decimal place — e.g. 23.1, never 231. No register anywhere should need a $\times 10$ mental correction to read.
Level / height	Computed and stored as float , full precision carried through the math (averaging, rate-error, any downstream use). Display only rounds to a whole millimetre — the HMI shows 184, the register underneath can legitimately hold 184.37.
Flow	Decimal, precision TBD by the flow sensor’s actual resolution — carried as float like level until a sensible display precision is chosen.

2.2 Comparison Tolerance

Set, not speculative. Floating-point equality/threshold compares ($D=$, $D\geq$) can drift right at a boundary due to representation error. All setpoint and rate compares throughout ADA 3.0 use a fixed tolerance band instead of bare equality:

$$\pm 1 \text{ mm for level,} \quad \pm 0.1 \text{ }^{\circ}\text{C for temperature.}$$

This is the one tolerance convention used everywhere a float is compared against a setpoint — Dynamic Water Target, Rate Error bounds, PO Timer thresholds, all of it.

3 Sensor Data Acquisition

Reads the raw hardware and converts to engineering units, using the same three physical paths as the ADA 2.0.1 front end, now pointer-driven instead of one hand-written conversion per channel.

3.1 Acquisition Loops

Loop	Channels	Source	Output
RTD → Temp	6	2 built-in PT100/analogue + 4-channel RTD/analogue card	6 decimal $^{\circ}\text{C}$ values, 1 dp
4AD → Level	4	4-channel analogue card (ultrasonic)	4 float mm values, base-referenced
1AD → Flow	1	Analogue flow input	1 decimal value

Each loop is one indexed block instead of N repeated rungs: a pointer walks the fixed-stride source registers, applies the shared scale/offset for that sensor type, and writes into the corresponding fixed-stride destination.

Design requirement carried from the review. Source and destination register blocks must be laid out at a **constant stride** from the start (e.g. every 2 registers) — the pointer loop depends on it, and changing stride later means touching every downstream reference.

3.2 Level Referencing

Level is now strict millimetres from the tank base — no more subtracting a raw ultrasonic distance from an “empty tank” reference inline in the math (the old D282/D284 pattern). The sensor’s mounting-height-to-base offset is folded into the conversion itself, so the output register is already a base-referenced height.

This does not remove the need for a measured offset at commissioning — it just moves that number to one place instead of leaving it implicit in a subtraction the reader has to reconstruct.

4 Sensor Averaging

Carried forward unchanged in concept from ADA 2.0.1 Section 5: a rolling 5-second block average (sum once per second, divide by 5 on the fifth addition, reset), applied to every channel that feeds a calculation. In ADA 3.0 this block is rewritten to run off the same pointer/stride convention as Acquisition, so the two blocks stay structurally aligned — one loop per sensor group, not one loop per channel.

5 Sensor Rate Error Calculations — Resolved

Flow is excluded entirely from this block. It is kept real-time — read and used directly for calculation/display, no averaging, no rate check. That leaves exactly **10 sensors** (6 temperature + 4 level), matching the original V1.1/V2.0 fault-analysis channel count and the ten channels ADA 2.0.1 Section 12.2 already identified as the Trend Threshold Supervision set.

Each of the 10 channels has its rate-of-change compared against **two bounds per channel** — a maximum rate and a minimum rate, a band rather than the old single-direction ratio check — all run through the pointer pattern established in Sections 3–4.

Channel group	Rate bound behaviour
6 temperature channels	Max/min rate hardwired in the PLC , not operator-facing.
4 level channels	Max/min rate accessible from the HMI — the runtime-adjustable half, the same role D500--D514 played in V1.1/V2.0, now exposed as an actual operator-facing setting rather than a register only visible in the ladder.

This mirrors the asymmetric tunability already established in ADA 2.0.1 Section 12.2: level channels are operator-adjustable because a bad level reading has immediate, severe knock-on effects (mass, T_f , tank scoring, priority); temperature channels are lower-consequence over the relevant timescale and stay on a fixed, PLC-hardwired bound.

6 Dynamic Water Target

Carried forward from ADA 2.0.1 Section 3 — the quadratic taper law is unchanged in shape: 70 °C below 50% tank fill, 40 °C above 90% fill.

$$T_{\text{target}}(d) = \begin{cases} 70, & d < 200 \\ 70 - \frac{3}{2560}(d - 200)^2, & 200 \leq d < 360 \\ 40, & d \geq 360 \end{cases}$$

What changed: setpoints are now expressed as real decimals per Section 2, rather than the old K700/K400 scaled integers. The knot points ($d = 200$, 50% full; $d = 360$, 90% full), the continuity proof at both boundaries, and the physical interpretation given in ADA 2.0.1 Section 3 all still apply without modification.

7 PO Designation

Flag	Meaning
M1 / M2 / M3	One bit per tank (T5/TA/TB/TC-style selection, whichever three tanks are eligible for PO designation).
M4	Aggregate “PO stored” flag: active if any of M1/M2/M3 is active, inactive if none are.

On selection, the chosen tank’s level and temperature are copied into a **separate PO data register set**, used only for PO-related calculation — this isolates the PO math from the live per-tank registers the rest of the program reads.

The currently-designated tank is excluded from ongoing need scoring while it holds the designation — see Section 9, which uses M1–M4 directly to implement that exclusion.

Mutual exclusion is enforced at the HMI. The operator physically cannot designate two tanks as PO at once; the input is locked to a single selection. Section 9’s “find the designated slot” step can rely on exactly one of M1/M2/M3 ever being active.

8 PO Timer Calculation — Resolved

Carried forward from the melting/liquid two-phase demand function established in ADA 2.0.1 Section 7 (D_{PO}), with the same watchdog-reset pattern (V1.1's T130) clearing stale demand on timeout, mirroring the Demand Watchdog described in ADA 2.0.1 Section 7.1.

The reference constants (melting point, target-temperature comparison — E135/E30 in the old ladder) **keep their values, unchanged** — only their number format moves to the Section 2 decimal convention. They stay **hardwired in the PLC program, not exposed to the HMI**: if the underlying physics ever needs revisiting, that is a programming change, not an operator setting.

9 Need Scheduler — Finalised

Gates the LSM / HSM / FSM modes in both automatic and manual control, same as ADA 2.0.1 Section 8. The part that was left vague in earlier revisions — how the Need Arbiter actually turns three tank scores into one decision — is resolved below, adopting the pointer-based approach.

9.1 Scoring — Confirmed Unchanged

Each of the three candidate tanks gets a score every scan:

$$\text{score} = 0.6 \times \text{Level} + 0.4 \times \text{Temperature}$$

identical to the formula fixed in ADA 2.0.1 Section 9.1. Level and temperature values feeding this carry the Section 2 tolerances (± 1 mm, ± 0.1 °C) — different sensors, same clean bounded calculation regardless of which physical sensor is behind each input.

9.2 Exclusion via a Sentinel, Not a Branch

The currently-designated PO tank (Section 7) should not compete for the arbiter's attention. Rather than adding a conditional skip inside the scoring loop, the three real scores are copied into a scratch array each scan and, **if M4 is active**, the designated tank's slot (found via M1/M2/M3) is overwritten with a sentinel value guaranteed to lose whichever comparison direction is active that scan. This keeps the arbiter's own loop completely uniform — it never needs to know *why* a slot is excluded, only that it cannot win.

9.3 One Loop, Direction Gated by M4

M4 state	Loop behaviour
Inactive (no tank designated — all 3 eligible)	Walk the scratch array once, indexed, tracking the running maximum and its index. Highest score wins, matching the “3 eligible → highest wins” rule from ADA 2.0.1 Section 9.2.1.
Active (one tank designated — 2 eligible)	Same loop, same array, but tracking the running minimum instead (the excluded slot is sentinel-forced high, so it can never be the minimum). Lowest score wins among the remaining two — urgency- prioritising, matching the inverted rule from ADA 2.0.1 Section 9.2.2.

This is the single mechanism ADA 2.0.1 needed two separate, hand-written comparison structures for. One indexed loop, one direction bit (M4), no per-combination branches — and it scales to a 4th candidate tank for free.

9.4 Tie-Break

Kept exactly as specified — no dedicated logic. The loop only replaces the current best when a candidate is *strictly* better, never on equal. That means whichever tank is encountered first in scan order naturally keeps its win on a tie — the same “lowest index wins ties” behaviour the old pairwise-compare chain produced implicitly, just inherited for free instead of coded.

9.5 On the Jitter

Accepted as-is. One optional, low-cost addition if the jitter ever looks worse than expected in commissioning — not required, easy to bolt on later without touching the arbiter itself: a short minimum-dwell timer on the winning index (e.g. 5–10 s) that has to elapse before a *new* index is allowed to take over, even if it wins the score comparison earlier. It is a pure debounce sitting downstream of the arbiter, independent of the 4-mode hysteresis (Section 8.1 of ADA 2.0.1) already doing the heavy lifting — skip it entirely if the existing distribution strategy holds up in practice.

9.6 Output

The winning tank index feeds both the branch-valve routing (replacing the old M101--M114 flag network) and, gated together with the Priority Scheduler’s demand-level flags, the LSM/HSM/FSM mode selection.

10 Oil Scheduler

Full redesign, replacing both the old oil-line logic and the D550 Modbus write-back mechanism. This section departs furthest from ADA 2.0.1's FIFO-queue description (Section 11 of that document); the request model below is the current, resolved design.

10.1 Per-Node Hardware

Two hardwired buttons per node — **Start/Stop** and **Reset** — plus an HMI numeric field for the requested oil amount.

10.2 Request Flow

1. Operator enters the amount on the HMI, presses **Start**.
2. PLC checks line occupancy.
 - **Busy:** the draining node's HMI shows ‘‘pumping...’’; the other two show ‘‘halted...’’. Presses on Start during this state are **not evaluated at all** — the PLC only re-checks once the line clears, not on every keypress.
 - **Free:** proceeds.
3. PLC polls Modbus for the requesting node, up to **5 tries**.
4. On success: numeric input flips to 0 (acknowledges the request was received), calculation completes, oil dispenses.
5. While dispensing, the line is locked — no other node's request is evaluated (no queue, no list — first request accepted when the line is free wins; a node waiting during “halted” simply re-tries Start once the line clears, nothing is remembered on its behalf).

10.3 Reset

Cancels **only the pressing node's own** pending/in-progress request — e.g. operator over-enters an amount, presses Stop, then Reset, done. It does not touch another node's active dispense.

10.4 5-Try Modbus Failure Handling

New in ADA 3.0 — closes the deadlock risk in the original design. Without this, a node stuck at 5 failed Modbus polls could leave the line lock held indefinitely with no recovery path.

1. **Release the line-occupancy lock immediately** on the 5th failed try, unconditionally, in the same rung that detects the failure — nothing else gates this.
2. **Distinct HMI message** for comms failure vs. “halted” (line busy elsewhere) — e.g. ‘‘No response, try again’’ — so the operator is not left thinking another node is draining when it is actually a comms fault.

3. **Don't clear the entered amount** on failure — it is a communication problem, not a data problem. A re-press of Start should retry without re-entry.
4. **Reset the try-counter to zero** when the lock is released, so the next attempt gets a full fresh 5 tries.
5. **Short cooldown** (1–2 s) before that node's Start is accepted again after a failure, to stop a flaky link from being hammered every scan with an identical, immediately-repeated failure.
6. **Persistent per-node failure counter**, separate from the retry-in-progress count and not self-clearing — same accumulate-and- reset-on-maintenance convention already used for oil totals (old D460--D462). Gives visibility into a node with a chronically flaky Modbus link instead of only ever seeing isolated retries.

Net result: three outcomes instead of two — **dispensed**, **busy-elsewhere**, or **comms-fault-retry** — each with its own HMI message, and no path where the line can lock up permanently.

11 Relationship to ADA 2.0.1's Fault and Oil-Line Scope

ADA 3.0 does not revisit the Fault Scheduler or the broader oil-distribution philosophy laid out in ADA Architecture 2.0.1 Sections 11–13 (single- destination full-flow dispensing, flowmeter-confirmed completion with no fault path, Trend Threshold Supervision, E-Stop gating, and operator override controls). Those remain governing at the architecture level. What this document adds is the ladder-level request/response mechanics for the oil line's node-facing HMI (Section 10 above) and the resolved rate-error and arbiter mechanisms (Sections 5 and 9) that ADA 2.0.1 left as architecture-only description without a specified implementation.

Open Items Before Freezing

None remaining. PO Timer constants keep their values unchanged (reformatted to Section 2's decimal convention), hardwired in the PLC program rather than exposed to the HMI — the last open item is closed. Every section in this document reflects a direct, confirmed design decision. **Ready to lock.**

Prepared by Sambridda Ranabhat · Mechatronics Engineering Pvt. Ltd. (MEPL), Kathmandu, Nepal

Based on the VCH Sizing Framework, Second Edition

DOI: 10.5281/zenodo.21009246

Supersedes ladder-level detail in ADA Architecture 2.0.1 (4 August 2026)